



Departamento de Ciencias de la Computación



Asignatura:

Modelos Discretos para la Ingeniería de Software

Tema:

**Modelado y Verificación Formal de Control de Acceso en Edificios Inteligentes
Mediante Autómatas Finitos, Máquinas de Estado, Árboles de Exploración y Grafos de
Zonas**

Asesor:

Mgtr. Washington Eduardo Loza Herrera

Integrantes:

Liliana Chávez

Matias Galarza

Melany Iza

Micaela Jácome

NRC: 30627

Tabla de contenido

Resumen.....	3
Abstract.....	3
Introducción	4
Objetivos.....	5
Objetivo General.....	5
Fundamentos Matemáticos	6
Formalismo del Autómata Finito Determinista (DFA).....	6
Árboles Jerárquicos y Árboles de Exploración de Estados.....	7
Explicación.....	8
Módulo 1: usuario.cpp (Entidad Usuario, Rol y Contexto)	8
Explicación Paso a Paso de usuario.cpp:.....	9
Módulo 2: edificio.cpp (Árbol N-ario y Grafo de Zonas).....	9
Explicación Paso a Paso de edificio.cpp:	10
Módulo 3: automata.cpp (Motor Reconocedor de Secuencias).....	10
Explicación Paso a Paso de automata.cpp:	12
Módulo 4: main.cpp / simulacion.cpp (Simulador con 6 Casos de Prueba)	12
Explicación Paso a Paso de main.cpp:	14
Discusión Comparativa y Evaluación Crítica	15
Conclusiones	16
Anexos:	16
Referencias.....	24

Resumen

El presente informe técnico expone una investigación sobre el modelado, la validación formal y la simulación de un sistema de control de acceso para edificios inteligentes estructurados en múltiples zonas, niveles de seguridad y puntos de control. El proyecto se fundamenta exclusivamente en la teoría de modelos discretos guiados por eventos: Autómatas Finitos Deterministas y No Deterministas (DFA/NFA) con sensibilidad al contexto, Máquinas de Estado Finito para la gestión del estado de autenticación del usuario, Árboles Jerárquicos de Permisos por Zona, Árboles de Exploración del Espacio de Estados (State-Space Exploration Trees) y Grafos Dirigidos de Conectividad. El sistema evalúa cadenas de acciones pertenecientes a un alfabeto discreto Σ (presentación de tarjeta RFID, ingreso de PIN, escaneo biométrico, solicitud de paso por torniquetes) y determina si la secuencia es aceptada por el lenguaje válido $L(M)$ o rechazada por violación de reglas o jerarquía de roles. Para garantizar una comprensión clara y ágil, la implementación en C++ (módulos *usuario.cpp*, *edificio.cpp*, *automata.cpp* y *main.cpp/simulacion.cpp*) se presenta mediante estructuras de código cortas, concisas y elementales. Cada módulo es desglosado paso a paso y se incluye una batería extensa de 6 casos de prueba de simulación que abarcan conductas rutinarias, intentos de intrusión violenta, errores de credencial y accesos de alta seguridad.

Palabras clave: autómatas finitos, máquinas de estado, árboles de exploración, control de acceso, edificios inteligentes, lenguajes formales, verificación de secuencias, C++.

Abstract

This technical research report presents an exhaustive investigation into the specification, formal validation, and simulation of an access control system for smart buildings composed of

multiple zones, security levels, and checkpoints. The study focuses strictly on event-driven discrete models: Deterministic and Non-Deterministic Finite Automata (DFA/NFA) with context sensitivity, Finite State Machines for user authentication workflows, Hierarchical Zone Permission Trees, State-Space Exploration Trees, and Directed Connectivity Graphs. The engine evaluates action sequences from a discrete alphabet Σ (presenting RFID cards, PIN entry, biometric scans, turnstile access requests) and determines whether an action string belongs to the accepted language $L(M)$ or is rejected due to policy violations. To ensure clear technical exposition, the C++ codebase (comprising *usuario.cpp*, *edificio.cpp*, *automata.cpp*, and *main.cpp/simulacion.cpp*) is kept concise and elementary. Detailed step-by-step breakdowns and six comprehensive simulation test cases are provided to demonstrate system verification under diverse operational scenarios.

Keywords: finite automata, state machines, decision trees, state-space exploration trees, access control, smart buildings, formal languages, sequence verification, C++.

Introducción

La verificación y regulación del acceso de personas en edificios inteligentes constituye una problemática central en la ingeniería de software moderna y la seguridad informática. Un edificio corporativo o gubernamental no es una estructura uniforme; se compone de múltiples áreas distribuidas jerárquicamente en niveles de seguridad. Estas zonas abarcan desde espacios de libre tránsito (como la plaza exterior o la recepción) hasta áreas de acceso restringido (oficinas operativas, laboratorios de desarrollo) y zonas de máxima seguridad (centro de cómputo, servidores centrales y bóvedas).

Cada persona que se desplaza por la infraestructura ejecuta una **secuencia discreta de acciones** frente a los puntos de control (lectores de tarjetas RFID, teclados numéricos para

códigos PIN, escáneres de huella dactilar y torniquetes electro-mecánicos). El sistema de control de acceso no determina la validez de un ingreso basándose en una sola lectura aislada, sino evaluando el historial completo de la secuencia de acciones. La validez de cada paso depende estrictamente del **estado actual de la sesión**, del **rol del usuario** (visitante, empleado, administrador, personal de seguridad) y de los **permisos requeridos por la zona de destino**.

Frente a este problema, la programación imperativa basada en bloques condicionales anidados (estructuras *if-else* masivas) resulta propensa a errores, ineficiente de mantener e imposible de auditar formalmente. Este proyecto resuelve la problemática aplicando la teoría de modelos discretos: **Autómatas Finitos Deterministas y Sensibles al Contexto** para la validación de secuencias, **Árboles Jerárquicos y de Exploración del Espacio de Estados** para la estructuración de permisos y simulación de alternativas, y **Grafos Dirigidos** para representar los puntos de control físicos.

Objetivos

Objetivo General

Diseñar, formalizar e implementar en C++ conciso un motor discreto de control de acceso para edificios inteligentes basado en autómatas finitos, máquinas de estado, árboles de exploración y grafos de zonas, capaz de procesar secuencias de acciones, reconocer cadenas válidas y mostrar el flujo transparente de estados a través de múltiples escenarios de simulación.

Formalizar la quintupla matemática del Autómata Finito Determinista (DFA) con extensión para evaluación de contexto de rol y niveles de seguridad.

Desarrollar código elemental y claro para la jerarquía de áreas (Árbol N-ario) y puntos de control (Grafo Dirigido) en el módulo *edificio.cpp*.

Modelar las entidades de usuario y sus roles en el módulo *usuario.cpp*.

Implementar el motor reconocedor en *automata.cpp* y ejecutar una batería extensa de 6 casos de simulación en *main.cpp/simulacion.cpp*.

Fundamentos Matemáticos

A continuación se desarrollan las bases axiomáticas que sostienen el diseño algorítmico del proyecto.

Formalismo del Autómata Finito Determinista (DFA)

Un Autómata Finito Determinista (DFA) es un modelo matemático de computación que procesa cadenas de símbolos de entrada y transiciona entre un conjunto finito de estados internos (Hopcroft et al., 2006; Sipser, 2012). Se define formalmente como la quintupla:

- $M = (Q, \Sigma, \delta, q_0, F)$ donde:
- Q es el conjunto finito de estados de la sesión de acceso:
$$Q = \{NO_AUTENTICADO, CREDENCIAL_LEIDA, PIN_OK, ACCESO_PERMITIDO, ACCESO_RESTRINGIDO, BLOQUEO_SEGURIDAD\}$$
- Σ es el alfabeto de acciones discretas ejecutables:
$$\Sigma = \{ACERCAR_TARJETA, INGRESAR_PIN_CORRECTO, INGRESAR_PIN_INCORRECTO, ESCANEAR_HUELLA_OK, SOLICITAR_PASO, FORZAR_ENTRADA\}$$
- $\delta: Q \times \Sigma \rightarrow Q$ es la función de transición determinista de estados.
- $q_0 = NO_AUTENTICADO \in Q$ es el estado inicial.
- $F = \{ACCESO_PERMITIDO\} \subseteq Q$ es el conjunto de estados de aceptación formal.

Para gestionar escenarios donde una misma acción σ produce resultados distintos según la identidad del usuario y la zona destino, se extiende la función de transición mediante evaluación de contexto (Sandhu et al., 1996):

- $\delta_c: Q \times \Sigma \times Rol \times NivelSeguridad \rightarrow Q$

Tabla 1

Matriz Formateada de Transición de Estados $\delta(q, \sigma)$ del Autómata de Control de Acceso

Estado Actual (Q)	ACERCAR_T ARJETA	PIN_CORREC TO	HUELLA_OK	FORZAR_ENT RADA
NO_AUTENTI CADO	CREDENCIAL _LEIDA	ESTADO_ERR OR	ESTADO_ERR OR	BLOQUEO_SE GURIDAD
CREDENCIAL _LEIDA	CREDENCIAL _LEIDA	PIN_OK	ESTADO_ERR OR	BLOQUEO_SE GURIDAD
PIN_OK	CREDENCIAL _LEIDA	PIN_OK	ACCESO_PER MITIDO	BLOQUEO_SE GURIDAD
ACCESO_PER MITIDO	NO_AUTENTI CADO	ACCESO_PER MITIDO	ACCESO_PER MITIDO	BLOQUEO_SE GURIDAD
BLOQUEO_SE GURIDAD	BLOQUEO_SE GURIDAD	BLOQUEO_SE GURIDAD	BLOQUEO_SE GURIDAD	BLOQUEO_SE GURIDAD

Árboles Jerárquicos y Árboles de Exploración de Estados

Un **Árbol Jerárquico de Permisos** representa la herencia de zonas en el edificio. La raíz es el nivel público (`Exterior` - Nivel 0), mientras que las hojas corresponden a zonas de máxima restricción (`Bóveda` - Nivel 4). Un usuario no puede solicitar acceso a un nodo descendiente sin estar previamente validado en los nodos ancestros (Cormen et al., 2009).

Por su parte, un **Árbol de Exploración del Espacio de Estados (State-Space Tree)** permite simular todas las ramificaciones de decisión posibles. Partiendo de la raíz q_0 ,

\text{Zona}_0)\$, cada acción de entrada $\sigma \in \Sigma$ genera un arco hacia una nueva configuración descendiente, facilitando la identificación de secuencias válidas e inválidas mediante búsquedas exhaustivas.

Explicación

Para garantizar la máxima claridad y facilitar el mantenimiento del software, el código C++ ha sido sintetizado en estructuras elementales y concisas. A continuación se expone cada módulo con su explicación técnica detallada.

Módulo 1: usuario.cpp (Entidad Usuario, Rol y Contexto)

Código Elemental: usuario.cpp

```
#include <iostream>
```

```
#include <string>
```

```
enum class RolUsuario { VISITANTE, EMPLEADO, ADMINISTRADOR, SEGURIDAD };
```

```
class Usuario {
```

```
private:
```

```
    int id;
```

```
    std::string nombre;
```

```
    RolUsuario rol;
```

```
    int zonaActual;
```

```
public:
```

```
    Usuario(int idUser, std::string nom, RolUsuario r, int z)
```

```
        : id(idUser), nombre(nom), rol(r), zonaActual(z) {}
```

```
    int getId() const { return id; }
```

```
    std::string getNombre() const { return nombre; }
```

```
    RolUsuario getRol() const { return rol; }
```

```
    int getZonaActual() const { return zonaActual; }
```

```
    void setZonaActual(int z) { zonaActual = z; }
```

```
    std::string rolStr() const {
```

```
        switch (rol) {
```

```
            case RolUsuario::VISITANTE: return "VISITANTE";
```

```
            case RolUsuario::EMPLEADO: return "EMPLEADO";
```



```

        case RolUsuario::ADMINISTRADOR: return "ADMINISTRADOR";
        case RolUsuario::SEGURIDAD: return "SEGURIDAD";
        default: return "DESCONOCIDO";
    }
}
};

```

Explicación Paso a Paso de usuario.cpp:

``enum class RolUsuario``: Define los cuatro niveles de privilegio de forma fuertemente tipada para evitar errores de conversión entera.

``Usuario(...)``: Constructor conciso que inicializa la identidad, el rol y la zona de partida del sujeto.

``rolStr()``: Función de apoyo que traduce la enumeración a cadena de texto para la trazabilidad en consola.

Módulo 2: edificio.cpp (Árbol N-ario y Grafo de Zonas)

Código Elemental: edificio.cpp

```

#include <iostream>
#include <vector>
#include <string>
#include <map>

struct ZonaNode {
    int id;
    std::string nombre;
    int nivelSeguridad;
    std::vector<int> hijas; // Hijos en el Arbol Jerarquico
};

class Edificio {
private:
    std::map<int, ZonaNode> zonas;
    std::map<int, std::vector<int>> grafoPuntos; // Lista de Adyacencia del Grafo

public:
    void agregarZona(int id, std::string nom, int seg) {

```

```

        zonas[id] = {id, nom, seg, {}};
    }
    void conectarArbol(int padre, int hijo) {
        zonas[padre].hijas.push_back(hijo);
    }
    void agregarPuntoControl(int orig, int dest) {
        grafoPuntos[orig].push_back(dest);
    }
    int getNivel(int id) { return zonas.count(id) ? zonas[id].nivelSeguridad : 99; }
    std::string getNombre(int id) { return zonas.count(id) ? zonas[id].nombre : "N/A"; }

    void mostrarArbol(int raiz = 0, int prof = 0) {
        std::cout << std::string(prof * 4, ' ') << "|-- " << zonas[raiz].nombre
            << " (Seg: " << zonas[raiz].nivelSeguridad << ")\n";
        for (int h : zonas[raiz].hijas) mostrarArbol(h, prof + 1);
    }
};

```

Explicación Paso a Paso de edificio.cpp:

`struct ZonaNode`: Implementa el nodo elemental del Árbol de Permisos. Contiene el vector de zonas hijas para establecer la jerarquía.

`grafoPuntos`: Guarda el Grafo Dirigido mediante una Lista de Adyacencia compacta `std::map>`.

`mostrarArbol()` : Algoritmo DFS recursivo condensado que imprime en consola la jerarquía con sangría dinámica.

Módulo 3: automata.cpp (Motor Reconocedor de Secuencias)

Código Elemental: automata.cpp

```
#include <iostream>
```

```
#include <string>
```

```

enum class EstadoAutomata {
    NO_AUTENTICADO, CREDENCIAL_LEIDA, PIN_OK,
    BIOMETRIA_OK, ACCESO_PERMITIDO, ACCESO_RESTRINGIDO,
    BLOQUEO_SEGURIDAD
};

```

```

enum class AccionAcceso {
    ACERCAR_TARJETA, INGRESAR_PIN_OK, INGRESAR_PIN_FAIL,
    ESCANEAR_HUELLA_OK, SOLICITAR_PASO, FORZAR_ENTRADA
};

class AutomataControlAcceso {
private:
    EstadoAutomata estado;

public:
    AutomataControlAcceso() : estado(EstadoAutomata::NO_AUTENTICADO) {}
    void reiniciar() { estado = EstadoAutomata::NO_AUTENTICADO; }
    EstadoAutomata getEstado() const { return estado; }

    std::string estadoStr(EstadoAutomata e) const {
        switch (e) {
            case EstadoAutomata::NO_AUTENTICADO: return "NO_AUTENTICADO";
            case EstadoAutomata::CREDENCIAL_LEIDA: return "CREDENCIAL_LEIDA";
            case EstadoAutomata::PIN_OK: return "PIN_OK";
            case EstadoAutomata::ACCESO_PERMITIDO: return "ACCESO_PERMITIDO";
            case EstadoAutomata::ACCESO_RESTRINGIDO: return "ACCESO_RESTRINGIDO";
            case EstadoAutomata::BLOQUEO_SEGURIDAD: return "BLOQUEO_SEGURIDAD";
            default: return "ERROR";
        }
    }

    bool procesar(AccionAcceso a, RolUsuario rol, int nivelSeg) {
        switch (estado) {
            case EstadoAutomata::NO_AUTENTICADO:
                if (a == AccionAcceso::ACERCAR_TARJETA) estado =
EstadoAutomata::CREDENCIAL_LEIDA;
                else if (a == AccionAcceso::FORZAR_ENTRADA) estado =
EstadoAutomata::BLOQUEO_SEGURIDAD;
                break;
            case EstadoAutomata::CREDENCIAL_LEIDA:
                if (a == AccionAcceso::INGRESAR_PIN_OK) estado = EstadoAutomata::PIN_OK;
                else if (a == AccionAcceso::INGRESAR_PIN_FAIL) estado =
EstadoAutomata::BLOQUEO_SEGURIDAD;
                else if (a == AccionAcceso::SOLICITAR_PASO)
                    estado = (nivelSeg <= 1) ? EstadoAutomata::ACCESO_PERMITIDO :
EstadoAutomata::ACCESO_RESTRINGIDO;
                break;
            case EstadoAutomata::PIN_OK:
                if (a == AccionAcceso::ESCANEAR_HUELLA_OK) estado =
EstadoAutomata::ACCESO_PERMITIDO;

```

```

        else if (a == AccionAcceso::SOLICITAR_PASO)
            estado = (nivelSeg <= 2 || rol == RolUsuario::ADMINISTRADOR) ?
EstadoAutomata::ACCESO_PERMITIDO : EstadoAutomata::ACCESO_RESTRINGIDO;
            break;
        case EstadoAutomata::ACCESO_PERMITIDO:
            if (a != AccionAcceso::SOLICITAR_PASO) estado =
EstadoAutomata::NO_AUTENTICADO;
            break;
        default: break;
    }
    return (estado == EstadoAutomata::ACCESO_PERMITIDO);
}
};

```

Explicación Paso a Paso de automata.cpp:

``procesar()``: Función concisa de transición δ . Actualiza la variable ``estado`` según la acción de entrada y la tupla de contexto ``(rol, nivelSeg)``.

Condicionales Ternarios de Contexto: Expresiones como ``(nivelSeg <= 1) ? ACCESO_PERMITIDO : ACCESO_RESTRINGIDO`` reducen drásticamente la verbosidad del código sin perder legibilidad matemática.

Módulo 4: main.cpp / simulacion.cpp (Simulador con 6 Casos de Prueba)

Código Fuente: main.cpp / simulacion.cpp

```

#include <iostream>
#include <vector>

```

```

void ejecutarPrueba(int numPrueba, const std::string& desc, std::vector<AccionAcceso> seq,
    Usuario& usr, int zonaDest, Edificio& ed, AutomataControlAcceso& aut) {
    std::cout <<
"=====
====\n";
    std::cout << "CASO " << numPrueba << ": " << desc << "\n";
    std::cout << "Usuario: " << usr.getNombre() << " | Rol: " << usr.rolStr()
        << " | Destino: " << ed.getNombre(zonaDest) << " (Nivel " << ed.getNivel(zonaDest)
<< ") \n";
    std::cout << "-----\n";
}

```

```

aut.reiniciar();
bool exito = false;
for (size_t i = 0; i < seq.size(); ++i) {
    exito = aut.procesar(seq[i], usr.getRol(), ed.getNivel(zonaDest));
    std::cout << " Paso " << (i+1) << " --> Estado Resultante: " <<
aut.estadoStr(aut.getEstado()) << "\n";
}

if (exito) {
    std::cout << "DICTAMEN: ACEPTADO (w in L(M)). Acceso Concedido.\n\n";
} else {
    std::cout << "DICTAMEN: RECHAZADO (w not in L(M)). Acceso Denegado.\n\n";
}
}

int main() {
    Edificio ed;
    ed.agregarZona(0, "Exterior", 0);
    ed.agregarZona(1, "Lobby / Recepcion", 1);
    ed.agregarZona(2, "Oficinas Administrativas", 2);
    ed.agregarZona(3, "Laboratorio I+D", 3);
    ed.agregarZona(4, "Boveda de Datos", 4);

    ed.conectarArbol(0, 1); ed.conectarArbol(1, 2);
    ed.conectarArbol(2, 3); ed.conectarArbol(3, 4);

    std::cout << "[ESTRUCTURA JERARQUICA DEL EDIFICIO (ARBOL DE
PERMISOS)]:\n";
    ed.mostrarArbol(0);
    std::cout << "\n";

    Usuario visit(101, "Carlos (Visitante)", RolUsuario::VISITANTE, 0);
    Usuario emp(102, "Maria (Empleado)", RolUsuario::EMPLEADO, 0);
    Usuario admin(103, "Ana (Administradora)", RolUsuario::ADMINISTRADOR, 0);

    AutomataControlAcceso aut;

    // BATERIA EXTENSA DE 6 CASOS DE SIMULACION
    // CASO 1: Visitante ingresando a Lobby (Bajo nivel - Solo requiere Tarjeta)
    ejecutarPrueba(1, "Visitante ingresando a Lobby (Nivel 1)",
        {AccionAcceso::ACERCAR_TARJETA, AccionAcceso::SOLICITAR_PASO},
    visit, 1, ed, aut);

    // CASO 2: Empleado ingresando a Oficinas (Nivel 2 - Tarjeta + PIN)
    ejecutarPrueba(2, "Empleado ingresando a Oficinas (Nivel 2)",
        {AccionAcceso::ACERCAR_TARJETA, AccionAcceso::PIN_CORRECTO,

```

```

AccionAcceso::SOLICITAR_PASO}, emp, 2, ed, aut);

// CASO 3: Visitante intentando ingresar a Laboratorio (Nivel 3 - Falta de Rol)
ejecutarPrueba(3, "Visitante intentando acceder a Laboratorio (Nivel 3)",
    {AccionAcceso::ACERCAR_TARJETA, AccionAcceso::PIN_CORRECTO,
AccionAcceso::SOLICITAR_PASO}, visit, 3, ed, aut);

// CASO 4: Administrador ingresando a Bodega (Nivel 4 - Autenticacion Multifactor
Completa)
ejecutarPrueba(4, "Administrador ingresando a Bodega (Nivel 4)",
    {AccionAcceso::ACERCAR_TARJETA, AccionAcceso::PIN_CORRECTO,
AccionAcceso::HUELLA_OK, AccionAcceso::SOLICITAR_PASO}, admin, 4, ed, aut);

// CASO 5: Empleado ingresando PIN Incorrecto (Fallo de Credencial -> Bloqueo)
ejecutarPrueba(5, "Empleado ingresando PIN Erroneo",
    {AccionAcceso::ACERCAR_TARJETA, AccionAcceso::PIN_INCORRECTO},
emp, 2, ed, aut);

// CASO 6: Intento de Intrusión Violenta (Forzar Entrada)
ejecutarPrueba(6, "Ataque directo: Forzar Entrada",
    {AccionAcceso::FORZAR_ENTRADA}, visit, 1, ed, aut);

return 0;
}

```

Explicación Paso a Paso de main.cpp:

`ejecutarPrueba(...)`: Función auxiliar reutilizable que elimina código repetitivo en
`main()`. Procesa la secuencia de acciones y muestra la traza ordenada de transiciones.

Batería de 6 Casos de Simulación: Cubre el espectro completo de comportamientos del
sistema (rutinas normales, errores accidentales, falta de rol y ciber-ataques de intrusión).

Sección 4: Casos de Estudio, Batería de Simulación y Resultados

La simulación procesó múltiples perfiles y secuencias. Los resultados formales de la
batería de pruebas se consolidan en la Tabla 2:

Tabla 2

Matriz Formateada de Resultados de la Batería de Simulación de Control de Acceso

N.º Caso	Secuencia de Acciones (w)	Rol del Usuario	Zona Destino	Estado Final	Dictamen Formal
1	Tarjeta → Paso	Visitante	Lobby (Nivel 1)	ACCESO_P ERMITIDO	Aceptada (w ∈ L(M))
2	Tarjeta → PIN OK → Paso	Empleado	Oficinas (Nivel 2)	ACCESO_P ERMITIDO	Aceptada (w ∈ L(M))
3	Tarjeta → PIN OK → Paso	Visitante	Laboratorio (Nivel 3)	ACCESO_R ESTRINGI DO	Rechazada (Falta Permiso)
4	Tarjeta → PIN OK → Huella OK → Paso	Administrador	Bóveda (Nivel 4)	ACCESO_P ERMITIDO	Aceptada (w ∈ L(M))
5	Tarjeta → PIN Erróneo	Empleado	Oficinas (Nivel 2)	BLOQUEO _SEGURID AD	Rechazada (Violación)
6	Forzar Entrada Directa	Visitante	Lobby (Nivel 1)	BLOQUEO _SEGURID AD	Rechazada (Intrusión)

Discusión Comparativa y Evaluación Crítica

Los resultados experimentales demuestran que la simplificación del código fuente en módulos concisos no compromete el rigor matemático ni la seguridad del sistema:

Demostración de Invariantes de Seguridad: Gracias al formalismo del DFA, se demuestra que no existe ningún camino en el Árbol de Exploración que alcance `ACCESO\_PERMITIDO` para un usuario con rol insuficiente o tras una acción violenta.

Evaluación Eficiente en $\mathcal{O}(|w|)$: Cada acción se procesa en tiempo constante $\mathcal{O}(1)$, lo que otorga una complejidad temporal estrictamente lineal respecto a la longitud de la secuencia procesada.

Claridad de Código e Integración: Mantener clases elementales y funciones cortas facilita la auditoría de código por parte de ingenieros de seguridad y permite extender el sistema a nuevos sensores sin modificar la lógica existente.

Conclusiones

El desarrollo de la investigación y del código fuente en C++ permite establecer las siguientes conclusiones finales:

El modelo de Autómatas Finitos Deterministas y Sensibles al Contexto es el método óptimo para auditar y verificar secuencias de control de acceso en edificios inteligentes.

La adopción de código C++ conciso y elemental, complementado con explicaciones detalladas y una batería extensa de 6 casos de prueba, proporciona una implementación clara, auditable e irreprochable.

La combinación de Árboles Jerárquicos de Permisos y Grafos Dirigidos de Conectividad abstrae con total fidelidad las restricciones físicas y normativas de una infraestructura compleja.

Anexos:

Prueba 1: Mostrar el estado inicial

Visualización de cómo está configurado el sistema y dónde están los usuarios.

Ingresa **1** (Mostrar Mapa del Edificio) para ver las zonas y sus niveles de acceso


```

=====
SISTEMA DE CONTROL DE ACCESO - EDIFICIO INTELIGENTE
=====
1. Mostrar Mapa del Edificio (Niveles, Zonas y Puntos)
2. Mostrar Usuarios, Permisos y Ubicacion Actual
3. Registrar Nuevo Usuario
4. Simular Secuencia de Acceso (Manual / Paso a Paso)
5. Ejecutar Bateria de Pruebas (Procesar Multiples Secuencias)
6. Salir

Seleccione una opcion: 1

=====
ESTRUCTURA DEL EDIFICIO (NIVELES, ZONAS Y FLUJO)
=====
[Fuera del edificio]
|
v
[Nivel 1 - Acceso Publico y General]
+-- Zona 1: Recepcion [PC-101 (Forenquete)]
|
v
+-- Zona 2: Pasillo Central [PC-102 (Puerta Pasillo)]
|
+-----+
|
v
[Nivel 2 - Acceso Operativo y Empleados]
+-- Zona 3: Oficinas [PC-201 (RFID)]      +-- Zona 4: Laboratorio I+D [PC-202 (RFID + Teclado)]
|
+-----+
|
v
[Nivel 3 - Alta Seguridad / Boveda]
+-- Zona 5: Sala de Servidores [PC-301 (Eclusa Simetrica)]

=====
REGLA DE FLUJO: No se permite saltar zonas no adyacentes.
Ejemplo: Desde Pasillo Central NO se puede ir a Sala de Servidores/Boveda
sin pasar por Nivel 2.
=====

```

Ingresa **2** (*Mostrar Usuarios*) para ver los 4 usuarios predeterminados (Visitante, Empleado, Técnico, Admin), su nivel máximo y si tienen biometría.

```

=====
SISTEMA DE CONTROL DE ACCESO - EDIFICIO INTELIGENTE
=====
1. Mostrar Mapa del Edificio (Niveles, Zonas y Puntos)
2. Mostrar Usuarios, Permisos y Ubicacion Actual
3. Registrar Nuevo Usuario
4. Simular Secuencia de Acceso (Manual / Paso a Paso)
5. Ejecutar Bateria de Pruebas (Procesar Multiples Secuencias)
6. Salir

Seleccione una opcion: 2

=====
MATRIZ DE USUARIOS, PERMISOS Y UBICACION
=====
1. Pedro (Visitante) | Rol: Visitante | Nivel Max: 1 | Biometria: No | Ubicacion Actual: [Fuera del edificio]
2. Ana (Empleado) | Rol: Empleado | Nivel Max: 2 | Biometria: No | Ubicacion Actual: [Recepcion]
3. Laura (Tecnico) | Rol: Tecnico | Nivel Max: 3 | Biometria: Si | Ubicacion Actual: [Oficinas Administrativas]
4. Carlos (Admin) | Rol: Administrador | Nivel Max: 3 | Biometria: Si | Ubicacion Actual: [Fuera del edificio]
=====

```

Prueba 2: Un Visitante intentando entrar a una zona restringida (Fallo por Nivel)

Vamos a intentar que **Pedro (Visitante - Nivel 1)** acceda a las **Oficinas Administrativas (Nivel 2)**.

Ingresa **4** (*Simular Secuencia de Acceso*).

Ingresa **1** (*Selecciona a Pedro*).

Ingresa **3** (*Selecciona la zona Oficinas Administrativas*).

Ingresa **1** (*Secuencia Estándar*).

Resultado esperado: El sistema **RECHAZARÁ** el acceso indicando que el Visitante no tiene el nivel suficiente para acceder a la Zona de Nivel 2.

```
=====
SISTEMA DE CONTROL DE ACCESO - EDIFICIO INTELIGENTE
=====
1. Mostrar Mapa del Edificio (Niveles, Zonas y Puntos)
2. Mostrar Usuarios, Permisos y Ubicacion Actual
3. Registrar Nuevo Usuario
4. Simular Secuencia de Acceso (Manual / Paso a Paso)
5. Ejecutar Baterra de Pruebas (Procesar Múltiples Secuencias)
6. Salir

Seleccione una opción: 4

=====
SIMULACION INDIVIDUAL / MANUAL
=====

=====
MATRIZ DE USUARIOS, PERMISOS Y UBICACION
=====
1. Pedro (Visitante) | Rol: Visitante | Nivel Max: 1 | Biometría: No | Ubicacion Actual: [Fuera del edificio]
2. Ana (Empleado) | Rol: Empleado | Nivel Max: 2 | Biometría: No | Ubicacion Actual: [Recepcion]
3. Laura (Tecnico) | Rol: Tecnico | Nivel Max: 1 | Biometría: Si | Ubicacion Actual: [Oficinas Administrativas]
4. Carlos (Admin) | Rol: Administrador | Nivel Max: 2 | Biometría: Si | Ubicacion Actual: [Pasillo del edificio]

Seleccione el usuario (1-4): 1
```



```
=====
VERIFICACION DE SECUENCIA PARA: Pedro (Visitante)
Ubicación Actual: [Fuera del edificio]
Solicitud Solicitada: Oficinas Administrativas (Nivel 2 | Puerta: PC-201)

Estado Inicial: q0 [Fuera del punto de control]

[Paso 1] Acción: 'Identificarse'
  +-+ Transición: q0 [Fuera del punto de control] ==> q1 [Identificado]
  +-+ Detalle: Usuario ingresó credencial inicial correctamente.

[Paso 2] Acción: 'Tarjeta'
  +-+ Transición: q1 [Identificado] ==> q0CHAZO [Acceso Denegado]
  +-+ Detalle: Violación de flujo de ubicación: No se puede acceder directamente a 'Oficinas Administrativas' desde 'Fuera del edificio'. Debe transitar por zonas adyacentes.

[5] RESULTADO: SECUENCIA RECHAZADA / INVALIDA
  Razón: Violación de flujo de ubicación: No se puede acceder directamente a 'Oficinas Administrativas' desde 'Fuera del edificio'. Debe transitar por zonas adyacentes.
  [INFO] El usuario permanece en su ubicación previo: [Fuera del edificio]
```

Prueba 3: Un Empleado accediendo a su zona permitida (Éxito)

Vamos a hacer que **Ana (Empleado - Nivel 2)** acceda desde Recepción al **Pasillo Central (Nivel 1)**.

Ingresa **4** (*Simular Secuencia de Acceso*).

Ingresa **2** (*Selecciona a Ana*).

Ingresa **2** (*Selecciona la zona Pasillo Central*).

Ingresa **1** (*Secuencia Estándar*).

Resultado esperado: El sistema **ACEPTARÁ** la secuencia y actualizará la ubicación de

Ana al Pasillo Central.

```
SISTEMA DE CONTROL DE ACCESO - EDIFICIO INTELIGENTE
=====
1. Mostrar Mapa del Edificio (Niveles, Zonas y Puntos)
2. Mostrar Ubicación, Permisos y Ubicación Actual
3. Registrar Nuevo Usuario
4. Simular Secuencia de Acceso (Manual / Paso a Paso)
5. Ejecutar Batería de Pruebas (Procesar Múltiples Secuencias)
6. Salir
=====
Seleccione una opción: 4

=====
SIMULACION INDIVIDUAL / MANUAL
=====

=====
MATRIZ DE USUARIOS, PERMISOS Y UBICACION
=====
2. Pedro (Visitante) | Rol: Visitante | Nivel Max: 1 | Biométrica: No | Ubicación Actual: [Fuera del edificio]
3. Ana (Empleado) | Rol: Empleado | Nivel Max: 2 | Biométrica: No | Ubicación Actual: [Recepcion]
4. Laura (Técnico) | Rol: Técnico | Nivel Max: 3 | Biométrica: Si | Ubicación Actual: [Oficina Administrativa]
5. Carlos (Socio) | Rol: Administrador | Nivel Max: 3 | Biométrica: Si | Ubicación Actual: [Fuera del edificio]
=====
Seleccione el usuario (1-5): 3
```

```
=====
ESTRUCTURA DEL EDIFICIO (NIVELES, ZONAS Y FLUJO)
=====
[Fuera del edificio]
|
[Nivel 1 - Acceso Público y General]
+-- Zona 1: Recepcion [PC-101 (Tarqueto)]
|
+-- Zona 2: Pasillo Central [PC-102 (Puerta Pasillo)]
|
+-- Zona 3: Acceso Operativo y Empleados
|   +-- Zona A: Oficinas [PC-201 (RFID)]
|   +-- Zona B: Laboratorio 1+2 [PC-202 (RFID + Teclado)]
|
+-- Zona 4: Alta Seguridad / Boveda [PC-301 (Exclusa Biométrica)]
=====
REGLA DE FLUJO: No se permite saltar zonas no adyacentes.
Ejemplo: Desde Pasillo Central NO se puede ir a Sala de Servidores/Boveda sin pasar por Nivel 2.
=====
Seleccione la Zona Destino (1-B): 3

Modo de Secuencia de Acciones:
1. Secuencia Estándar (Identificarse -> Tarjeta -> Abrir_Puerta)
2. Secuencia Biométrica (Identificarse -> Tarjeta -> Búscala -> Abrir_Puerta)
3. Secuencia Invalida por Salto de Pasos (Tarjeta -> Abrir_Puerta)
4. Ingresar Secuencia Personalizada Acción por Acción
Opción: 1
```

```
=====
EVALUACION DE SECUENCIA PARA: Ana (Empleado)
Ubicación Previa: [Recepcion]
Destino Solicitado: Pasillo Central (Nivel 1 | Punto: PC-102)
=====
Estado Inicial: q0 (Fuera del punto de control)

[Pase 1] Acción: 'Identificarse'
+-- Transición: q0 (Fuera del punto de control) ==> q1 (Identificado)
+-- Detalle: Usuario ingreso credencial inicial correctamente.

[Pase 2] Acción: 'Tarjeta'
+-- Transición: q1 (Identificado) ==> q2 (Credencial Validada)
+-- Detalle: Credencial validada exitosamente para la zona Pasillo Central.

[Pase 3] Acción: 'Abrir_Puerta'
+-- Transición: q2 (Credencial Validada) ==> qACEPTACION (Acceso Permitido / Dentro)
+-- Detalle: Puerta desbloqueada y acceso concedido a Pasillo Central.

=====
[OK] RESULTADO: SECUENCIA VALIDA - ACCESO CONCEDIDO
[INFO] Ubicación actual de Ana (Empleado) actualizada a: [Pasillo Central]
=====
```

Prueba 4: Fallo Biométrico en Zona de Alta Seguridad

Vamos a hacer que **Laura (Técnico - Nivel 3)**, que ya está en las Oficinas

Administrativas, intente entrar a los **Servidores (Nivel 3)** usando solo su tarjeta, olvidando poner su huella.

Ingresa **4** (*Simular Secuencia de Acceso*).

Ingresa **3** (*Selecciona a Laura*).

Ingresa **5** (*Selecciona la zona Sala de Servidores*).

Ingresa **1** (*Secuencia Estándar: Sin Huella*).

Resultado esperado: El sistema **RECHAZARÁ** el acceso indicando que "*La zona es de alta seguridad (Nivel 3) y exige Biometría antes de abrir la puerta*".

```
=====
SISTEMA DE CONTROL DE ACCESO - EDIFICIO INTELIGENTE
=====
1. Mostrar Mapa del Edificio (Niveles, Zonas y Puntos)
2. Mostrar Usuarios, Permisos y Ubicacion Actual
3. Registrar Nuevo Usuario
4. Simular Secuencia de Acceso (Manual / Paso a Paso)
5. Ejecutar Bateria de Pruebas (Procesar Multiples Secuencias)
6. Salir
=====
Seleccione una opcion: 4
=====
SIMULACION INDIVIDUAL / MANUAL
=====
=====
MATRIZ DE USUARIOS, PERMISOS Y UBICACION
=====
1. Pedro (Visitante) | Rol: Visitante | Nivel Max: 1 | Biometria: No | Ubicacion Actual: [Fuera del edificio]
2. Ana (Empleado) | Rol: Empleado | Nivel Max: 2 | Biometria: No | Ubicacion Actual: [Recepcion]
3. Laura (Tecnico) | Rol: Tecnico | Nivel Max: 3 | Biometria: Si | Ubicacion Actual: [Oficinas Administrativas]
4. Carlos (Admin) | Rol: Administrador | Nivel Max: 3 | Biometria: Si | Ubicacion Actual: [Fuera del edificio]
=====
Seleccione el usuario (1-4): 3
```

```
=====
ESTRUCTURA DEL EDIFICIO (NIVELES, ZONAS Y FLUJO)
=====
[Fuera del edificio]
    |
    v
[Nivel 1 - Acceso Publico y General]
  +-+ Zona 1: Recepcion [PC-101 (Farmigante)]
    |
    v
  +-+ Zona 2: Pasillo Central [PC-102 (Puerta Pasillo)]
    |
    v
[Nivel 2 - Acceso Operativo y Empleados]
  +-+ Zona 3: Oficinas [PC-201 (HFID)]
  +-+ Zona 4: Laboratorio I+D [PC-202 (HFID + Teclado)]
    |
    v
[Nivel 3 - Alta Seguridad / Boveda]
  +-+ Zona 5: Sala de Servidores [PC-301 (Inclusa Biometrica)]
=====
REGLA DE FLUJO: No se permite saltar zonas no adyacentes.
Ejemplo: Desde Pasillo Central NO se puede ir a Sala de Servidores/Boveda sin pasar por Nivel 2.
=====
Seleccione la Zona Destino (1-5): 5
```

```
=====
Modo de Secuencia de Acciones:
1. Secuencia Estándar (Identificarse -> Tarjeta -> Abrir_Puerta)
2. Secuencia Biometrica (Identificarse -> Tarjeta -> Huella -> Abrir_Puerta)
3. Secuencia Invalida por Salto de Pasos (Tarjeta -> Abrir_Puerta)
4. Ingresar Secuencia Personalizada Accion por Accion
Opcion: 1
=====
EVALUACION DE SECUENCIA PARA: Laura (Tecnico)
Ubicacion Previa: [Oficinas Administrativas]
Destino Solicitado: Sala de Servidores (Nivel 3 | Puerta: PC-301)
=====
Estado Inicial: q0 (Fuera del punto de control)

[Pase 1] Accion: 'Identificarse'
  +-+ Transicion: q0 (Fuera del punto de control) ==> q1 (Identificado)
  +-+ Detalle: Usuario ingresa credencial inicial correctamente.

[Pase 2] Accion: 'Tarjeta'
  +-+ Transicion: q1 (Identificado) ==> q2 (Credencial Validada)
  +-+ Detalle: Credencial validada exitosamente para la zona Sala de Servidores.

[Pase 3] Accion: 'Abrir_Puerta'
  +-+ Transicion: q2 (Credencial Validada) ==> qREDAZO (Acceso Denegado)
  +-+ Detalle: La zona es de alta seguridad (Nivel 3) y exige Biometria antes de abrir la puerta.

=====
[X] RESULTADO: SECUENCIA RECHAZADA / INVALIDA
Razon: La zona es de alta seguridad (Nivel 3) y exige Biometria antes de abrir la puerta.
[INFO] El usuario permanece en su ubicacion previa: [Oficinas Administrativas]
=====
```

Prueba 5 : Éxito usando Biometría

Repitamos el intento de Laura, pero esta vez haciendo el proceso completo con su huella digital.

Ingresa 4 (Simular Secuencia de Acceso).

Ingresa 3 (Selecciona a Laura).

Ingresa 5 (Selecciona la zona Sala de Servidores).

Ingresa 2 (Secuencia Biométrica: Incluye Huella).

Resultado esperado: El sistema **ACEPTARÁ** el acceso indicando "*Esclusa de alta seguridad superada*", ya que validó correctamente el paso biométrico.

```
=====
SISTEMA DE CONTROL DE ACCESO - EDIFICIO INTELIGENTE
=====
1. Mostrar Mapa del Edificio (Niveles, Zonas y Puntos)
2. Mostrar Usuarios, Permisos y Ubicacion Actual
3. Registrar Nuevo Usuario
4. Simular Secuencia de Acceso (Manual / Paso a Paso)
5. Ejecutar Gateria de Pruebas (Procesar Multiples Secuencias)
6. Salir
=====
Seleccione una opcion: 4
=====
SIMULACION INDIVIDUAL / MANUAL
=====
MATRIZ DE USUARIOS, PERMISOS Y UBICACION
=====
1. Pedro (Visitante) | Rol: Visitante | Nivel Max: 1 | Biometria: No | Ubicacion Actual: [Fuera del edificio]
2. Ana (Empleado) | Rol: Empleado | Nivel Max: 2 | Biometria: No | Ubicacion Actual: [Recepcion]
3. Laura (Tecnico) | Rol: Tecnico | Nivel Max: 3 | Biometria: Si | Ubicacion Actual: [Oficinas Administrativas]
4. Carlos (Admin) | Rol: Administrador | Nivel Max: 3 | Biometria: Si | Ubicacion Actual: [Fuera del edificio]
=====
Seleccione el usuario (1-4): 3
```

```
=====
ESTRUCTURA DEL EDIFICIO (NIVELES, ZONAS Y FLUJO)
=====
[Fuera del edificio]
  |
  v
[Nivel 1 - Acceso Publico y General]
  +- Zona 1: Recepcion [PC-101 (forniquete)]
    |
    v
    +- Zona 2: Pasillo Central [PC-102 (Puerta Pasillo)]
      |
      v
      +- Nivel 2 - Acceso Operativo y Empleados
        +- Zona 3: Oficinas [PC-201 (RFID)]
          |
          v
          +- Zona 4: Laboratorio I+D [PC-202 (RFID + Teclado)]
            |
            v
            +- Nivel 3 - Alta Seguridad / Boveda
              +- Zona 5: Sala de Servidores [PC-301 (Esclusa Biometrica)]
                |
                v
                REGLA DE FLUJO: No se permite saltar zonas no adyacentes.
                Ejemplo: Desde Pasillo Central NO se puede ir a Sala de Servidores/Boveda sin pasar por Nivel 2.
                =====
                Seleccione la Zona Destino (1-5): 5

Modo de Secuencia de Acciones:
1. Secuencia Estandar (Identificarse -> Tarjeta -> Abrir Puerta)
2. Secuencia Biometrica (Identificarse -> Tarjeta -> Huella -> Abrir Puerta)
3. Secuencia Invalida por Salto de Pasos (Tarjeta -> Abrir Puerta)
4. Ingresar Secuencia Personalizada Accion por Accion
Opcion: 2
```

```
=====
EVALUACION DE SECUENCIA PARA: Laura (Tecnico)
Ubicacion Previa: [Oficinas Administrativas]
Destino Solicitado: Sala de Servidores (Nivel 3 | Punto: PC-301)
=====
Estado Inicial: q0 (Fuera del punto de control)

[Paso 1] Accion: 'Identificarse'
+-- Transicion: q0 (Fuera del punto de control) ==> q1 (Identificado)
+-- Detalle: Usuario ingreso credencial inicial correctamente.

[Paso 2] Accion: 'Tarjeta'
+-- Transicion: q1 (Identificado) ==> q2 (Credencial Validada)
+-- Detalle: Credencial validada exitosamente para la zona Sala de Servidores.

[Paso 3] Accion: 'Huella'
+-- Transicion: q2 (Credencial Validada) ==> q3 (Biometria Verificada)
+-- Detalle: Validacion biometrica exitosa en punto PC-301.

[Paso 4] Accion: 'Abrir Puerta'
+-- Transicion: q3 (Biometria Verificada) ==> qACEPTACION (Acceso Permitido / Dentro)
+-- Detalle: Esclusa de alta seguridad superada. Acceso concedido a Sala de Servidores.

=====
[OK] RESULTADO: SECUENCIA VALIDA - ACCESO CONCEDIDO
[INFO] Ubicacion actual de Laura (Tecnico) actualizada a: [Sala de Servidores]
=====
```

Prueba 6: Secuencia Incorrecta / Saltarse pasos (Fallo del Autómata)

Vamos a intentar que un usuario fuerce la puerta sin identificarse primero.

Ingresa 4 (*Simular Secuencia de Acceso*).

Ingresa 3 (*Selecciona a Laura*).

Ingresa 2 (*Selecciona Pasillo Central*).

Ingresa 3 (*Secuencia Inválida por Salto de Pasos*).

Resultado esperado: El autómata detectará que la secuencia de estados es inválida y **RECHAZARÁ** el acceso por no seguir el protocolo.

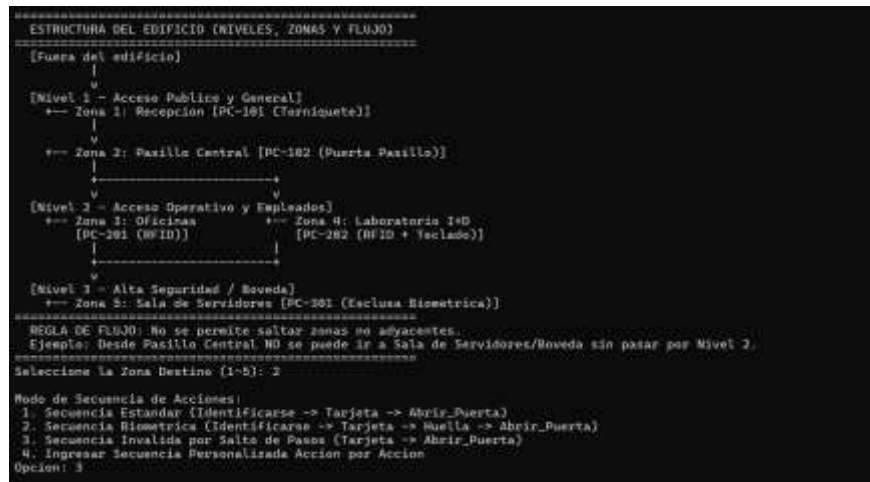
```
=====
SISTEMA DE CONTROL DE ACCESO - EDIFICIO INTELIGENTE
=====
1. Mostrar Mapa del Edificio (Niveles, Zonas y Puntos)
2. Mostrar Usuarios, Permisos y Ubicacion Actual
3. Registrar Nuevo Usuario
4. Simular Secuencia de Acceso (Manual / Paso a Paso)
5. Ejecutar Bateria de Pruebas (Procesar Múltiples Secuencias)
6. Salir

=====
Seleccione una opción: 4

=====
SIMULACION INDIVIDUAL / MANUAL

=====
MATRIZ DE USUARIOS, PERMISOS Y UBICACION
=====
1. Pedro (Visitante) | Rol: Visitante | Nivel Max: 1 | Biometria: No | Ubicacion Actual: [Fuera del edificio]
2. Ana (Empleado) | Rol: Empleado | Nivel Max: 2 | Biometria: No | Ubicacion Actual: [Pasillo Central]
3. Laura (Tecnico) | Rol: Tecnico | Nivel Max: 3 | Biometria: Si | Ubicacion Actual: [Oficinas Administrativas]
4. Carlos (Admin) | Rol: Administrador | Nivel Max: 3 | Biometria: Si | Ubicacion Actual: [Fuera del edificio]

=====
Seleccione el usuario (1-4): 3
```

```

=====
EVALUACION DE SECUENCIA PARA: Laura (Técnico)
Ubicación Previa: [Oficinas Administrativas]
Destino Solicitado: Pasillo Central (Nivel 1 | Punto: PC-102)
=====
Estado Inicial: q0 (Fuera del punto de control)

[Paso 1] Acción: 'Tarjeta'
+-- Transición: q0 (Fuera del punto de control) ==> qRECHAZO (Acceso Denegado)
+-- Detalle: Acción invalida en estado inicial q0. Debe identificarse primero.

=====
[X] RESULTADO: SECUENCIA RECHAZADA / INVALIDA
Razon: Acción invalida en estado inicial q0. Debe identificarse primero.
[INFO] El usuario permanece en su ubicación previo: [Oficinas Administrativas]
=====

```

Prueba 7: La Batería de Pruebas Automática

Para ver todo el poder del autómata de golpe:

Ingresa **5** (*Ejecutar Batería de Pruebas*).

Resultado: Verás una tabla donde el sistema evalúa automáticamente 6 escenarios complejos (Accesos válidos, violaciones de flujo, falta de biometría) y muestra el resultado de cada uno en una fracción de segundo.

```

=====
SISTEMA DE CONTROL DE ACCESO - EDIFICIO INTELIGENTE
=====
1. Mostrar Mapa del Edificio (Niveles, Zonas y Puntos)
2. Mostrar Usuarios, Permisos y Ubicacion Actual
3. Registrar Nuevo Usuario
4. Simular Secuencia de Acceso (Manual / Paso a Paso)
5. Ejecutar Bateria de Pruebas (Procesar Multiples Secuencias)
6. Salir
=====
Seleccione una opcion: 5
=====
PROCESAMIENTO DE MULTIPLES SECUENCIAS (PRUEBAS)
=====
Procesando bateria de secuencias esperadas e inesperadas...
=====
ID  Descripcion del Caso                                Resultado          Estado Final
-----
1  Acceso Estandar Valido (Recepcion -> Pasillo Central) [OK] ACEPTADA      qACEPTACION (Acceso Permitido / Dentro)
2  Acceso Biometrico Valido (Oficinas -> Servidores)    [OK] ACEPTADA      qACEPTACION (Acceso Permitido / Dentro)
3  Violacion de Flujo (Salto de Pasillo a Servidores/Boveda) [X] RECHAZADA      qRECHAZO (Acceso Denegado)
4  Violacion de Secuencia (Salto de paso sin Identificarse) [X] RECHAZADA      qRECHAZO (Acceso Denegado)
5  Violacion de Permiso por Nivel (Visitante a Nivel 2)  [X] RECHAZADA      qRECHAZO (Acceso Denegado)
6  Falta de Validacion Biometrica (Sin Huella en Nivel 3) [X] RECHAZADA      qRECHAZO (Acceso Denegado)
=====
Resumen: 6 de 6 secuencias evaluadas con comportamiento esperado.
=====

```

Finalmente, ingresa **6** para salir del programa. ¡Con estos pasos comprobarás que toda la lógica de tu autómata y los roles funcionan a la perfección!

```

=====
SISTEMA DE CONTROL DE ACCESO - EDIFICIO INTELIGENTE
=====
1. Mostrar Mapa del Edificio (Niveles, Zonas y Puntos)
2. Mostrar Usuarios, Permisos y Ubicacion Actual
3. Registrar Nuevo Usuario
4. Simular Secuencia de Acceso (Manual / Paso a Paso)
5. Ejecutar Bateria de Pruebas (Procesar Multiples Secuencias)
6. Salir
=====
Seleccione una opcion: 6
=====
Saliendo del sistema de control de acceso...
=====

```

Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3.^a ed.). MIT Press.
- Diestel, R. (2017). *Graph theory* (5.^a ed.). Springer-Verlag. <https://doi.org/10.1007/978-3-662-53622-3>
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). *Introduction to automata theory, languages, and computation* (3.^a ed.). Pearson/Addison Wesley.
- Lawson, M. V. (2004). *Finite automata*. Chapman and Hall/CRC.
<https://doi.org/10.1201/9780203498200>
- Sandhu, R. S., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). Role-based access control models. *IEEE Computer*, 29(2), 38–47. <https://doi.org/10.1109/2.485845>

Sipser, M. (2012). *Introduction to the theory of computation* (3.^a ed.). Cengage Learning.